BOARD VERIFICATION SIGN-OFF
Week 5 · D Flip-Flop & 8-bit Shift Register with Enable · Altera DE0 · CLOCK_50, KEY[0]=rst_n, KEY[1]=en, SW[0]=serial_in → LEDG[7:0]

Instructor signature: _____ Date: _____ Marks awarded: _____ / 10

Lab 5 — Sequential Logic: Flip-Flops & Registers

Name: _____	**Student ID:** _____

Learning Objectives	By the end of this lab you will be able to: (1) describe the three elements of a sequential circuit — next-state logic, state register, and output logic; (2) explain why the D flip-flop is the one implemented in FPGA hardware; (3) write a D flip-flop and an N-bit register with correct reset and initialization behavior; (4) distinguish blocking from non-blocking assignment and avoid the bugs each one causes; (5) use a Verilog parameter to reuse the same module with different settings, such as register width or a round's key; (6) trace values through a multi-stage (pipelined) sequential circuit and wire together separate module files; and (7) design, synthesize, and verify an 8-bit shift register with enable on the DE0 board.

Part A · Sequential Circuit Elements (Exercise 1)

MATCH	**Exercise 1 — Elements of a Sequential Circuit**	**10 pts**

Match each term below to its correct description. Write the matching letter (A–E) in the blank before each numbered description.

Terms:

A. Next-State Logic B. State Register C. Output Logic D. Feedback Path E. Clock

#	Match (A–E)	Description
1		Combinational logic that computes what the new state should be next.
2		The bank of flip-flops that stores the current state — the only place memory exists.
3		The connection where the state register's output loops back to become an input of the next-state logic.
4		Combinational logic that turns the state (and sometimes the inputs) into the outputs.
5		The periodic signal whose edge is the only moment the state register is allowed to change.

Part B · Initialization, Reset & Debugging (Exercise 2)

DEBUG	Exercise 3 — Find All 5 Bugs	10 pts

The two modules below are meant to compile cleanly and behave as a simple registered pass-through: `dff_reg` resets to 0 and otherwise loads `d` every clock edge; `Top` instantiates it and drives `data_out`. As written, there are **exactly 5 bugs** spread across both modules — a syntax error, a sensitivity-list mistake, a reset-polarity mistake, a blocking-assignment mistake, and a wiring mistake. Find all 5.

```
 1  module dff_reg (
 2    input        clk, rst_n, en
 3    input  [7:0] d,
 4    output reg [7:0] q
 5  );
 6
 7    always @(posedge clk, rst_n) begin
 8      if (rst_n)
 9        q <= 8'b0;
10      else if (en)
11        q = d;
12    end
13  endmodule
14
15  module Top (
16    input        clk, rst_n,
17    input  [7:0] data_in,
18    output [7:0] data_out
19  );
20
21    wire [7:0] q_wire;
22
23    dff_reg u1 (.clk(clk), .rst_n(rst_n), .en(1'b1), .d(data_in), .Q(q_wire));
24
25    assign data_out = q_wire;
26
27  endmodule
```

Fill in one row per bug (order doesn't matter). **2 points each**: 1 for correctly locating and describing the bug, 1 for the correct fix.

#	Line / Location	What is wrong?	Corrected code
1			
2			
3			
4			
5			

Part C · Blocking vs Non-blocking Assignment (Exercise 3)

PREDICT	Exercise 4 — Blocking vs Non-blocking: Two Versions, Five Questions	10 pts

Below are **two versions** of the same 4-stage register chain — `confusing_reg` uses blocking assignment (`=`) throughout, `correct_reg` uses non-blocking (`<=`) throughout. For all questions, assume the same starting condition: `rst_n = 1` (not resetting), `en = 1`, `d = 1`, and `q1 = q2 = q3 = q4 = 0` before the first clock edge.

```
module confusing_reg (              // ALL BLOCKING
  input       clk, rst_n, en,
  input       d,
  output reg  q_out
);
  reg q1, q2, q3, q4;
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      q1 = 1'b0; q2 = 1'b0; q3 = 1'b0; q4 = 1'b0;
    end
    else if (en) begin
      q1 = d;
      q2 = q1;
      q3 = q2;
      if (q3) q4 = 1'b1;
      else    q4 = q3;
    end
    q_out = q4;
  end
endmodule
```

```
module correct_reg (                // ALL NON-BLOCKING
  input       clk, rst_n, en,
  input       d,
  output reg  q_out
);
  reg q1, q2, q3, q4;
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      q1 <= 1'b0; q2 <= 1'b0; q3 <= 1'b0; q4 <= 1'b0;
    end
    else if (en) begin
      q1 <= d;
      q2 <= q1;
      q3 <= q2;
      if (q3) q4 <= 1'b1;
      else    q4 <= q3;
    end
    q_out <= q4;
  end
endmodule
```

Fill in the value of every signal for each situation below (6 pts for the table).

Signal	confusing_reg (blocking) — after edge 1	correct_reg (non-blocking) — after edge 1	correct_reg (non-blocking) — after edge 2
q1			
q2			
q3			
q4			
q_out			

Part D · Shift Register: Trace the Button Presses (Exercise 5)

PREDICT	Exercise 5 — What Does q_reg Become After Each Press?	10 pts

Here is the complete, working `shift_reg8.v`, already wired to the DE0 (`CLOCK_50` → clk, `KEY[0]` → rst_n, `KEY[1]` → en, `SW[0]` → serial_in, `LEDG[7:0]` → q).

```
module shift_reg8 (
  input        clk, rst_n, en, serial_in,
  output [7:0] q
);
  reg [7:0] q_reg;

  always @(posedge clk or negedge rst_n)
    if (!rst_n)
      q_reg <= 8'b0;
    else if (en)
      q_reg <= {q_reg[6:0], serial_in};

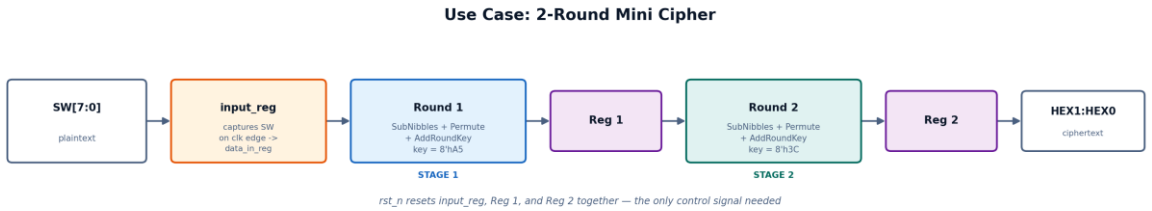
  assign q = q_reg;
endmodule
```

Someone presses `KEY[0]` once first (reset, so `q_reg = 00000000`). Then they press `KEY[1]` five times, holding `SW[0]` at the value shown **before** each press. Fill in `q_reg` (8-bit binary) **after** each press.

KEY[1] press #	SW[0] before this press	q_reg after this press (8-bit binary)
1	1	
2	0	
3	1	
4	1	
5	0	

Part G · Multi-Stage Sequential Circuits (Exercises 6–8)

Use case: a **2-round mini block cipher** built the same way real AES is, just far smaller. Every round applies **SubNibbles** + **Permute** + **AddRoundKey** (see behavior box below). `input_reg` captures SW[7:0] into the clk domain as `data_in_reg` before Round 1 ever sees it. **rst_n** is the only control signal — no extra buttons needed. The ciphertext shows in hex on `HEX1:HEX0`.



The S-box (given, same table used in every round):

in	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
out	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

Program the full design onto the DE0.

DE0 Pin	Connects to	Purpose
CLOCK_50	clk	The board's real 50 MHz oscillator — no manual stepping
SW[7:0]	data_in (via input_reg)	8-bit plaintext byte to encrypt
KEY[0]	rst_n (active-low)	Reset all registers
HEX1:HEX0	reg2_q (via hex7seg, given)	Ciphertext, in hex

Confirm the steady state. With `CLOCK_50` driving `clk` at 50 MHz, all 3 register stages settle in about 60 nanoseconds — **far too fast to see**, unlike the button-stepped shift register lab earlier this week. Press `KEY[0]` once to reset. For **each** row below, set `SW` to the value shown and read `HEX1:HEX0` — it will already show the settled result. Only the first row can be checked against Exercise 6 (that's the value you traced by hand); just record what you observe for the rest.

round_function.v (given — do not modify. Read it once; you'll instantiate it, not rewrite it):

```
module round_function #(
  parameter [7:0] ROUND_KEY = 8'h00
) (
  input  [7:0] data_in,
  output [7:0] data_out
);
  wire [3:0] sub_hi, sub_lo;
  wire [7:0] substituted, permuted;

  function [3:0] sbox;
    input [3:0] x;
    begin
      case (x)
        ____________________________________________________________________
        ____________________________________________________________________
        ____________________________________________________________________
        ____________________________________________________________________
      endcase
```

```
    end
  endfunction

  assign sub_hi      = sbox(data_in[7:4]);       // SubNibbles, hi nibble
  assign sub_lo      = ____________________       // SubNibbles, lo nibble
  assign substituted = {sub_hi, sub_lo};
  assign permuted    = ______________________________// Permute
  assign data_out    = permuted ^ _________;      // AddRoundKey
endmodule
```

How the combinational Round Function behaves:

Round Function	**SubNibbles:** look up each 4-bit nibble in the S-box above — a pure table lookup, same input nibble always gives the same output nibble.
	Permute: swap the two nibbles — `output = {input[3:0], input[7:4]}`. No bit's value changes, only its position.
	AddRoundKey: bitwise XOR with that round's fixed key — flips exactly the bits where the key has a 1.
	
	Worked example — Round 1 on input 0xC3 with key 0xA5:
	`Step 1  SubNibbles:`
	a) Write the input in binary and split it into two 4-bit nibbles:
	`0xC3 = 1100 0011  ->  hi nibble = 1100 (0xC),`
	`lo nibble = 0011 (0x3)`
	b) Look up each nibble in the S-box table above (find the column matching the nibble, read the value in the 'out' row):
	`S-box[0xC] = 0x5  (column C -> out = 5)`
	`S-box[0x3] = 0x1  (column 3 -> out = 1)`
	c) Put the two substituted nibbles back together, hi nibble first:
	`{0x5, 0x1}  =  0101 0001  =  0x51`
	`Step 2  Permute:`
	swap nibbles of 0x51 -> 0x15
	`Step 3  AddRoundKey:`
	0x15 XOR 0xA5 = 0001 0101 XOR 1010 0101 = 1011 0000 = **0xB0**
	0xB0 is exactly the Reg 1 value you should get at Edge 2 of Exercise 6 — use this same 3-step process for Round 2 with key 0x3C.

FILL-IN-BLANK	**Exercise 7 — Complete the Support Modules**	**10 pts**

You are given the completed `round_function.v` and `hex7seg.v` (do not modify). Complete the three modules below — they're all used by the top-level design.

7A — input_reg.v: capture SW into the pipeline (3 blanks, 3 pts)

```
module input_reg (
  input        clk, rst_n,
  input  [7:0] sw,
  output reg [7:0] data_in_reg
);
  always @(_____________ clk or negedge rst_n)
    if (!rst_n) data_in_reg <= _______________;
    else        data_in_reg _______________ sw;
endmodule
```

7B — register.v: the reusable N-bit register (3 blanks, 3 pts)

```
module register #(parameter W = 8) (
  input            clk, rst_n,
  input  [W-1:0]   d,
  output reg [W-1:0] q
);
  always @(posedge clk or negedge rst_n)
    if (_____________) q <= _____________;
    else              q <= _____________;
endmodule
```

7C — hex7seg.v (given — do not modify. Turns one 4-bit hex digit into a 7-segment pattern):

```
module hex7seg (
  input  [3:0] value,
  output reg [6:0] seg   // active-low, segments {g,f,e,d,c,b,a}
);
  always @(*) begin
    case (value)
      4'h0: seg = 7'b1000000;   4'h1: seg = 7'b1111001;
      4'h2: seg = 7'b0100100;   4'h3: seg = 7'b0110000;
      4'h4: seg = 7'b0011001;   4'h5: seg = 7'b0010010;
      4'h6: seg = 7'b0000010;   4'h7: seg = 7'b1111000;
      4'h8: seg = 7'b0000000;   4'h9: seg = 7'b0010000;
      4'hA: seg = 7'b0001000;   4'hB: seg = 7'b0000011;
      4'hC: seg = 7'b1000110;   4'hD: seg = 7'b0100001;
      4'hE: seg = 7'b0000110;   4'hF: seg = 7'b0001110;
    endcase
  end
endmodule
```

7D — mini_cipher_top.v: wire everything together (4 blanks, 4 pts)

```
module mini_cipher_top (
  input        CLOCK_50,
  input  [7:0] SW,
  input        KEY0,         // rst_n
  output [6:0] HEX1, HEX0
);
  wire clk = CLOCK_50, rst_n = KEY0;
  wire [7:0] data_in_reg, round1_c, reg1_q, round2_c, reg2_q;

_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________
_____________________________________________________________________

endmodule
```

SW (data_in)	HEX1:HEX0 observed (hex)
11000011 (0xC3)	
00000000 (0x00)	
11111111 (0xFF)	
01011101 (0x5D)	
10100101 (0xA5)	